ARQUITECTURA N CAPAS

Introducción

DISEÑO

PROCESO DE PLANIFICACIÓN Y MODELADO DE LA ESTRUCTURA, COMPONENTES, INTERFACES Y DATOS DE UNA APLICACIÓN ANTES DE SU CODIFICACIÓN

ARQUITECTURA

DISEÑO ESTRUCTURAL DE ALTO NIVEL DE UN SISTEMA, DEFINIENDO SUS COMPONENTES, RELACIONES Y PRINCIPIOS DE FUNCIONAMIENTO

INFRAESTRUCTURA

CONJUNTO DE COMPONENTES LÓGICOS —SISTEMAS OPERATIVOS, BASES DE DATOS, REDES, MIDDLEWARE Y APLICACIONES— QUE SOPORTAN LA OPERACIÓN, GESTIÓN Y ENTREGA DE SERVICIOS DE TI EN UNA ORGANIZACIÓN

PATRON DE DISEÑO

SOLUCIONES REUTILIZABLES, PROBADAS Y ESTANDARIZADAS PARA PROBLEMAS COMUNES EN EL DESARROLLO DE SOFTWARE, ACTUANDO COMO PLANTILLAS CONCEPTUALES.

VER MAS EN HTTPS://REFACTORING.GURU/ES/DESIGN-PATTERNS

ARQUITECTURA N CAPAS

ARQUITECTURAS DE SOFTWARE

CLIENTE-SERVIDOR

Ventajas

- Centraliza datos y control
- Escalable horizontalmente
- Separacion de operaciones

Desventajas

- Dependencia al servidor
- Dependencia a la latencia de red

MONOLITICA

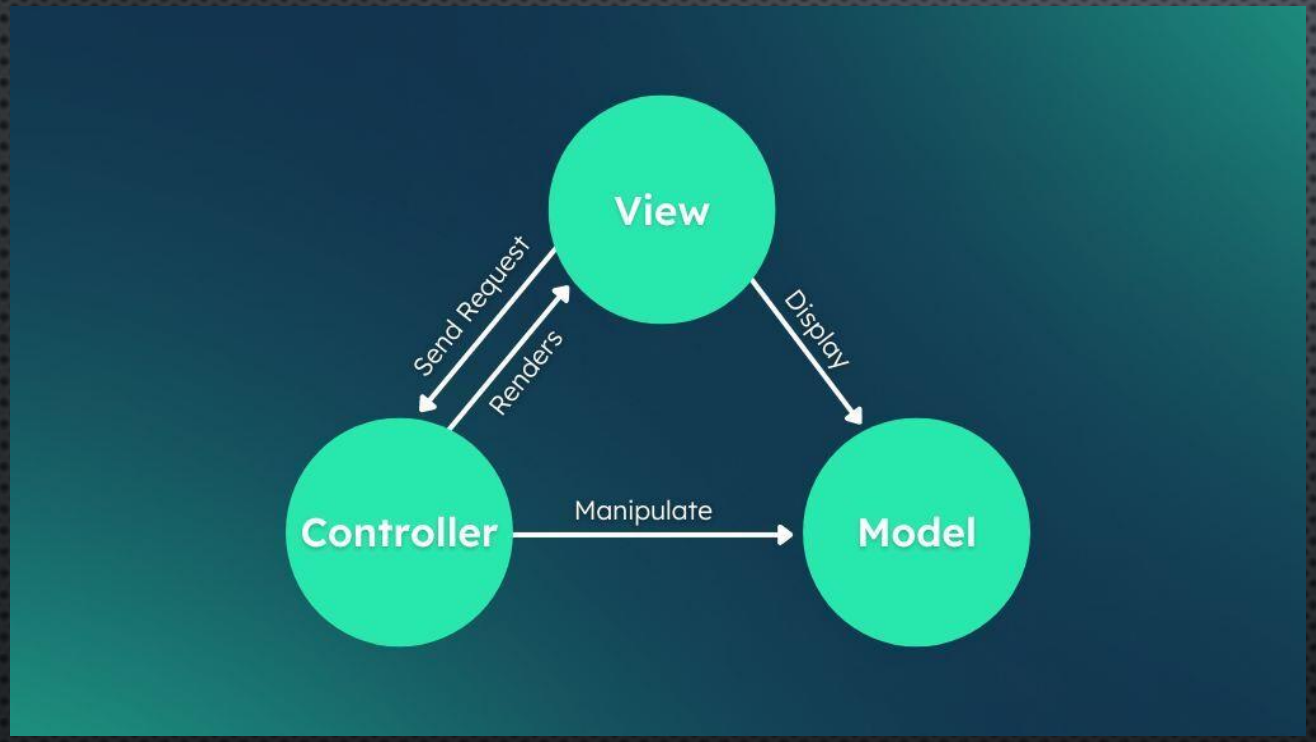
Ventajas

- Desarrollo rapido
- Comunicacion entre modulos rapida
- Despliegue simple

Desventajas

- Perdida de control de versiones
- Dificil de escalar
- Dificil de mantener en el tiempo

MVC



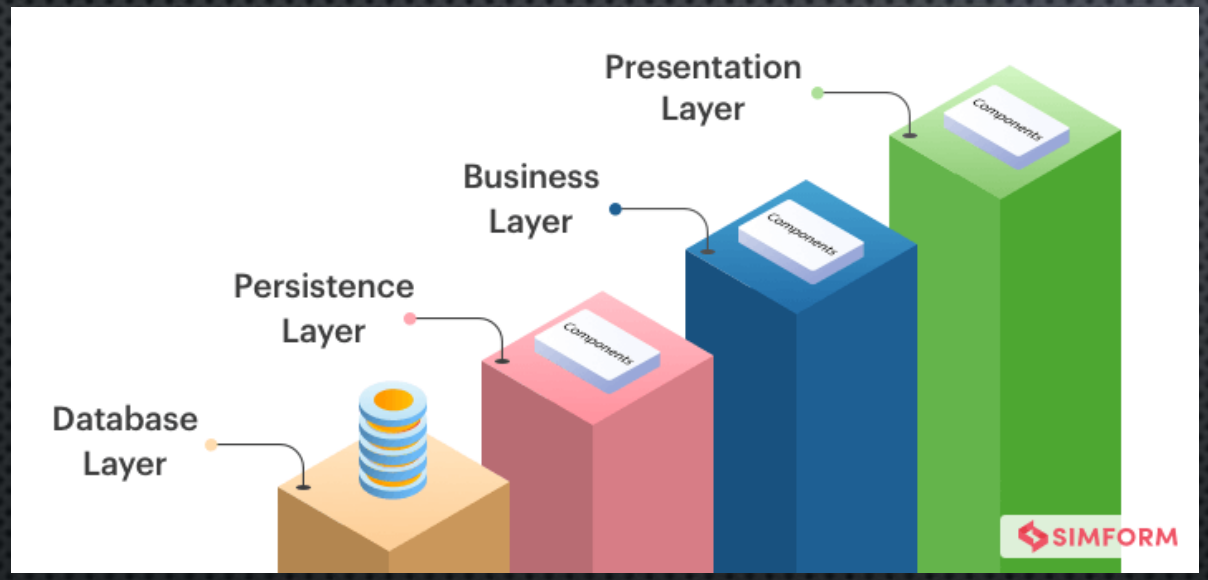
Ventajas

- Separacion de responsabilidades
- Colaboracion entre miembros

Desventajas

- Repeticion de codigo alta
- No es ideal para escalar sistemas grandes

NCAPAS



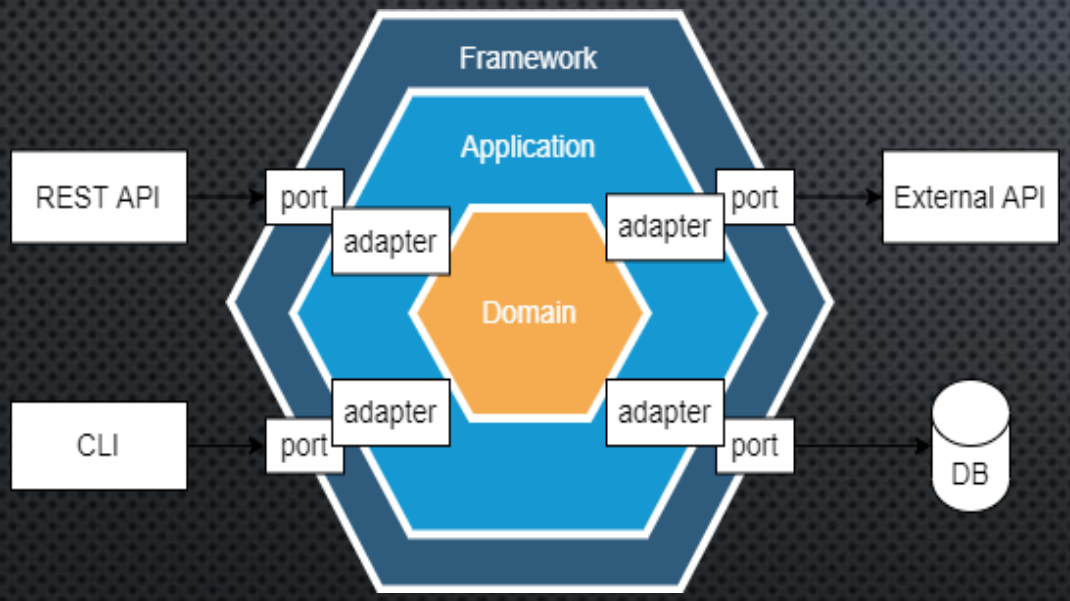
Ventajas

- Estructura clara
- Mantenible y escalable
- Reutilizacion de capas
- Versatilidad

Desventajas

- Suceptible a baja cohesion
- Complejidad aumenta al aumentar capas
- Dependencias entre capas

HEXAGONAL



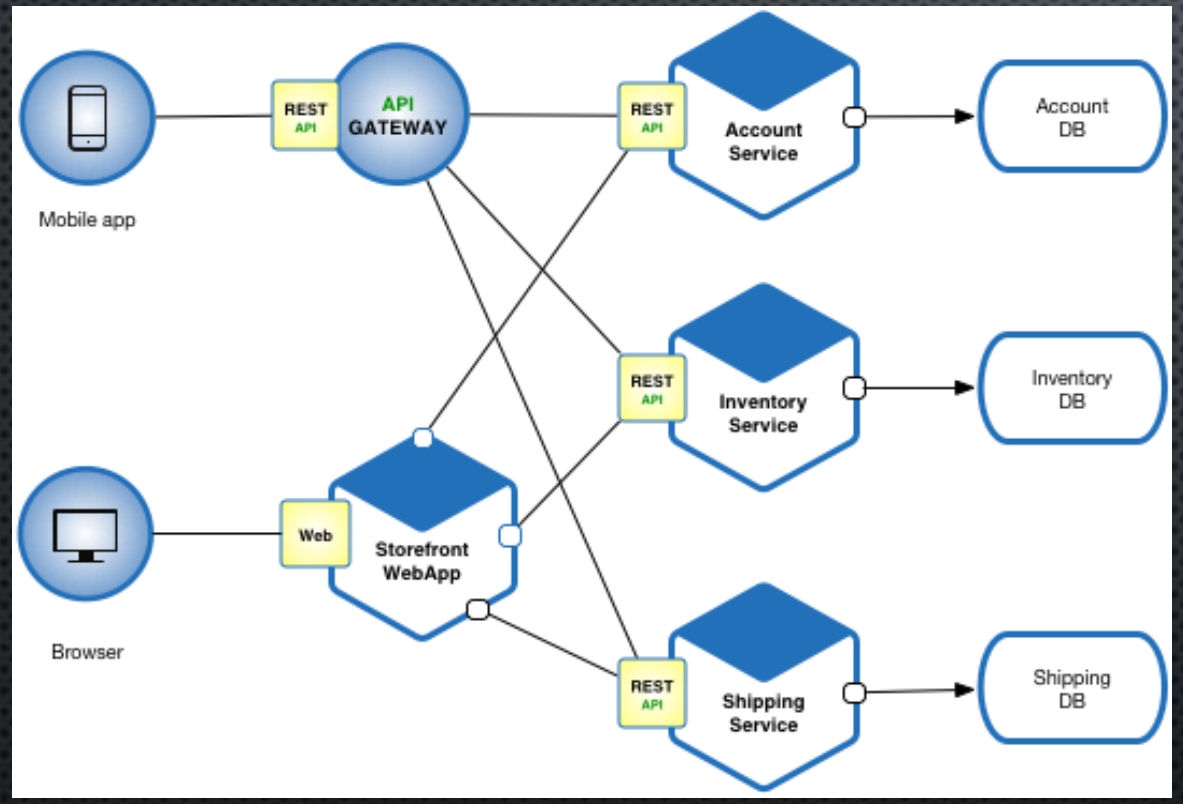
Ventajas

- Testeable, mantenible, escalable
- Independencia y desacoplamiento de modulos

Desventajas

- Complejidad mas alta
- Excesiva en la mayoria de proyectos

MICROSERVICIOS



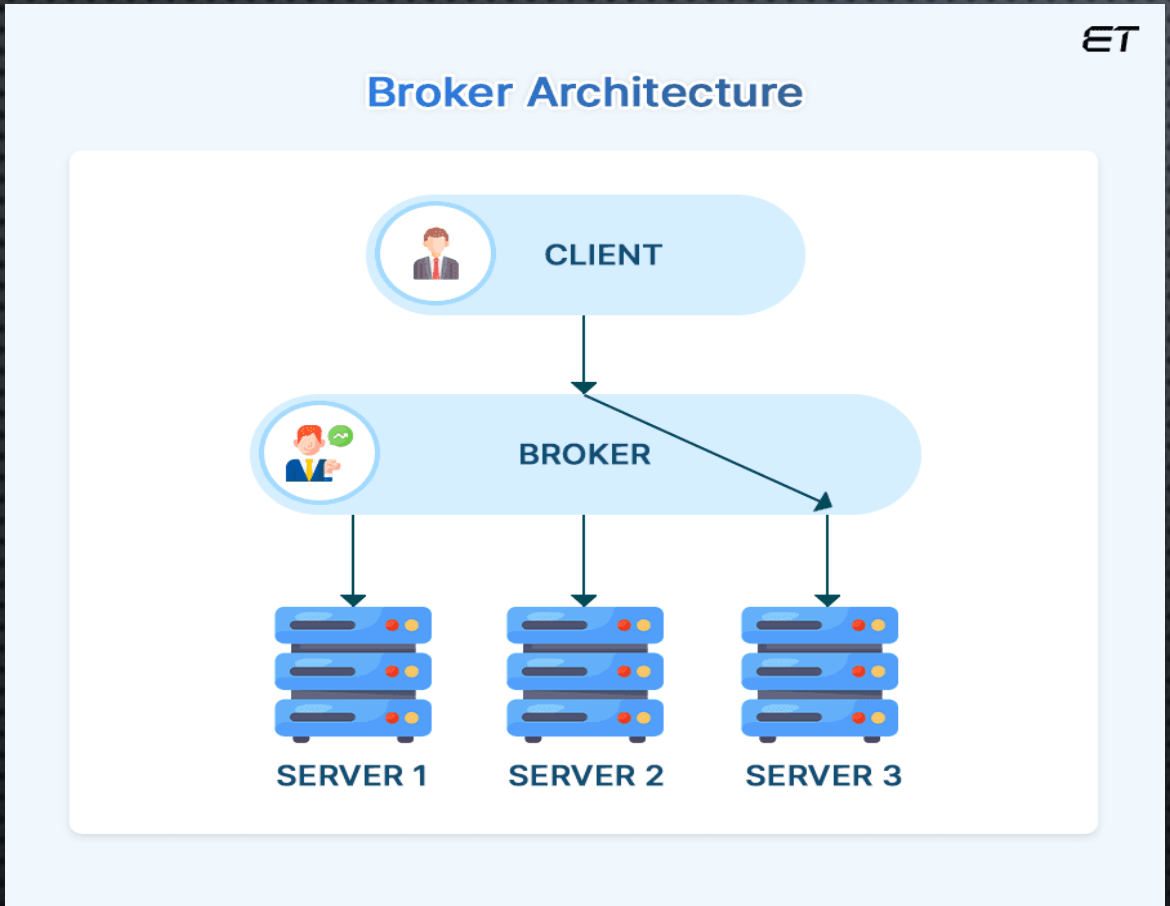
Ventajas

- Alto desacoplamiento
- Alta escalabilidad
- Independencia de servicios

Desventajas

- Infraestructura compleja y cara de mantener
- Complejidad de red a medida escala
- Dependencia a la red

BROKER



Ventajas

- Buen manejo de carga
- Escalable horizontalmente

Desventajas

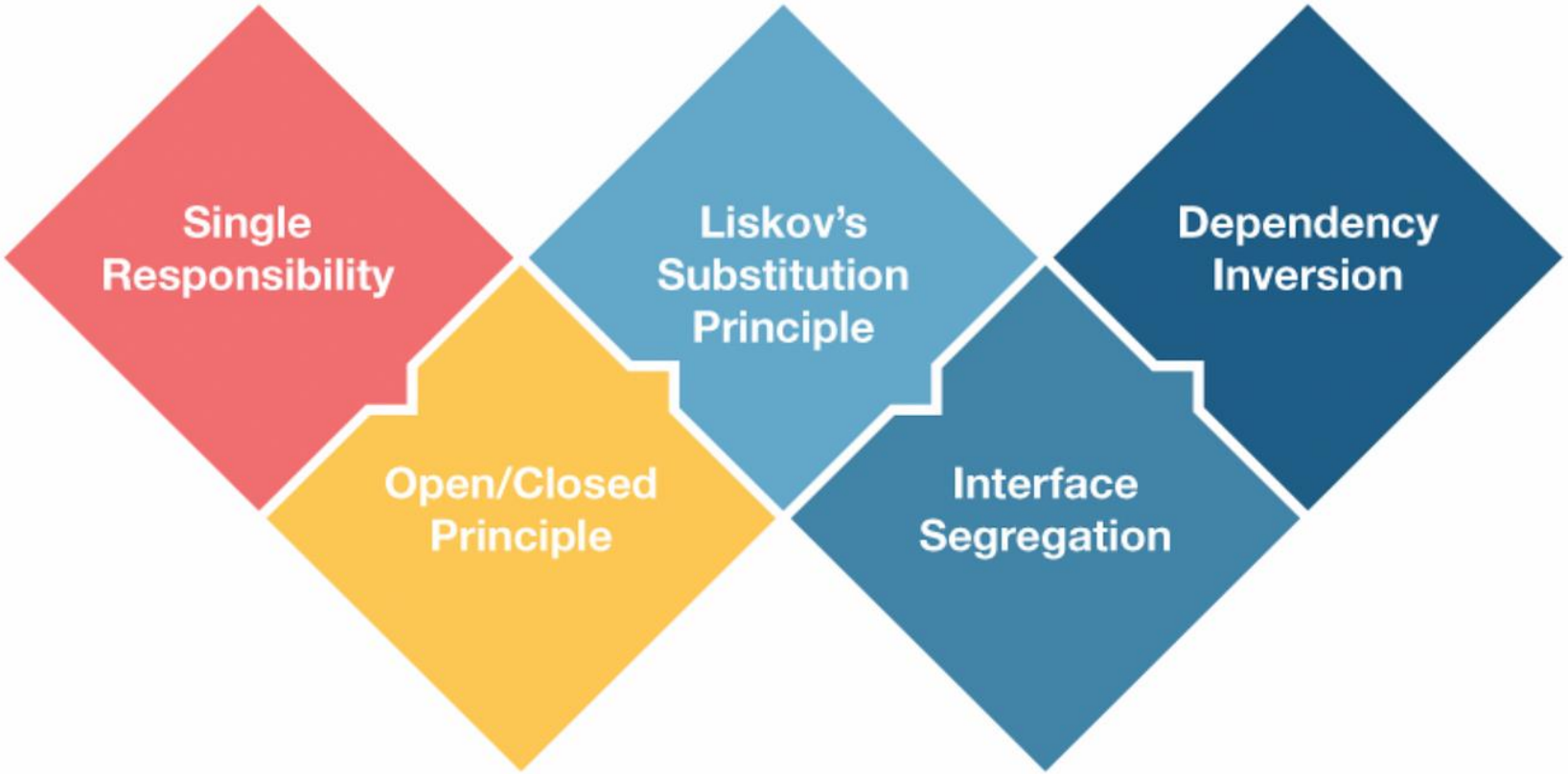
- Complejidad de configuracion
- Dependencia a un punto de entrada

ARQUITECTURA N CAPAS

Principios de programacion

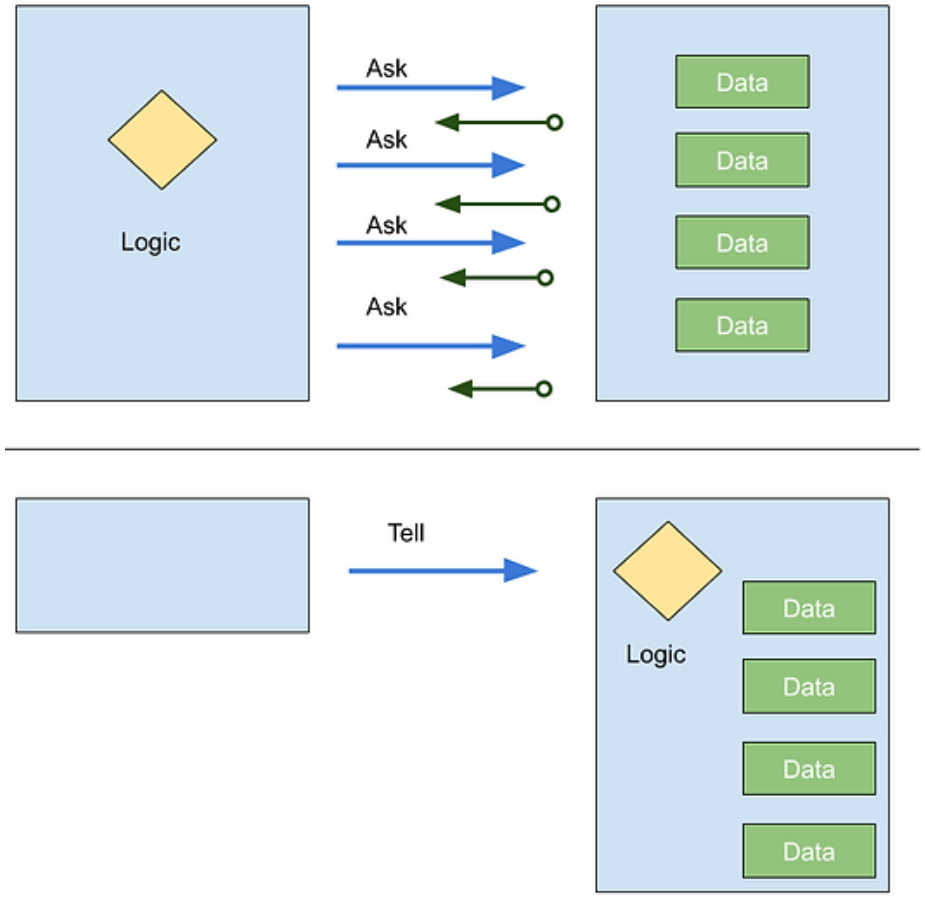
4 Concepts of OOP

S.O.L.I.D.


Single Responsibility
Open/Closed Principle
Liskov's Substitution Principle
Interface Segregation
Dependency Inversion

$$S \leq T \rightarrow (\forall x{:}T.\, \phi(x) \rightarrow \forall y{:}S.\, \phi(y))$$

GOOD PRACTICES



TELL, DON'T ASK (TDA)

YOU AIN'T GONNA NEED IT (YAGNI)

FAIL FAST

KEEP IT SHORT AND SIMPLE (KISS)

DON'T REPEAT YOURSELF (DRY)

DOCUMENT YOUR CODE (DYC)?

COMPOSITION OVER INHERITANCE

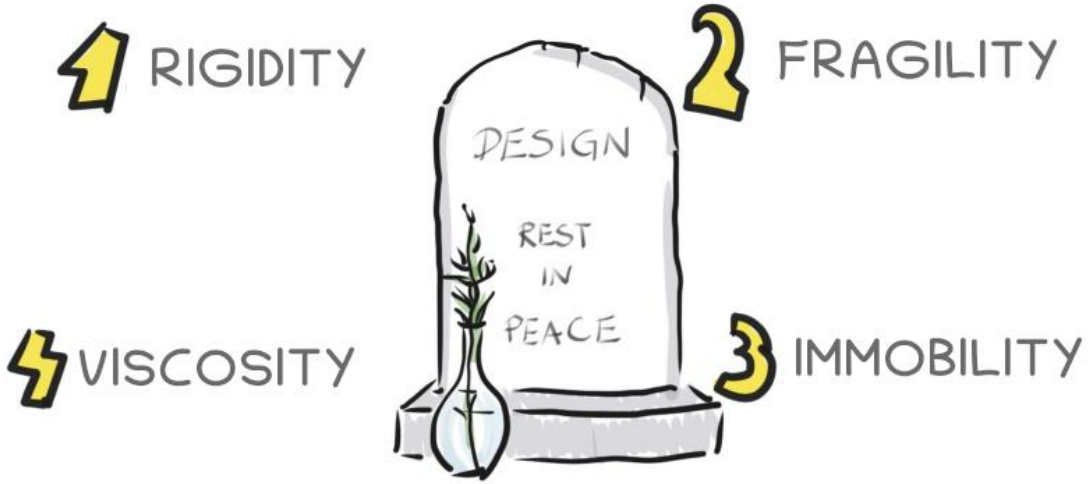
QUÉ QUEREMOS LOGRAR?

- Readability
- Maintainability
- Collaboration
- Reliability
- Efficiency
- Testability
- Security
- Modularity
- Clarity

Metrics

- Code coverage
- Cohesion
- Coupling
- Cyclomatic Complexity
- Bugs per line of code

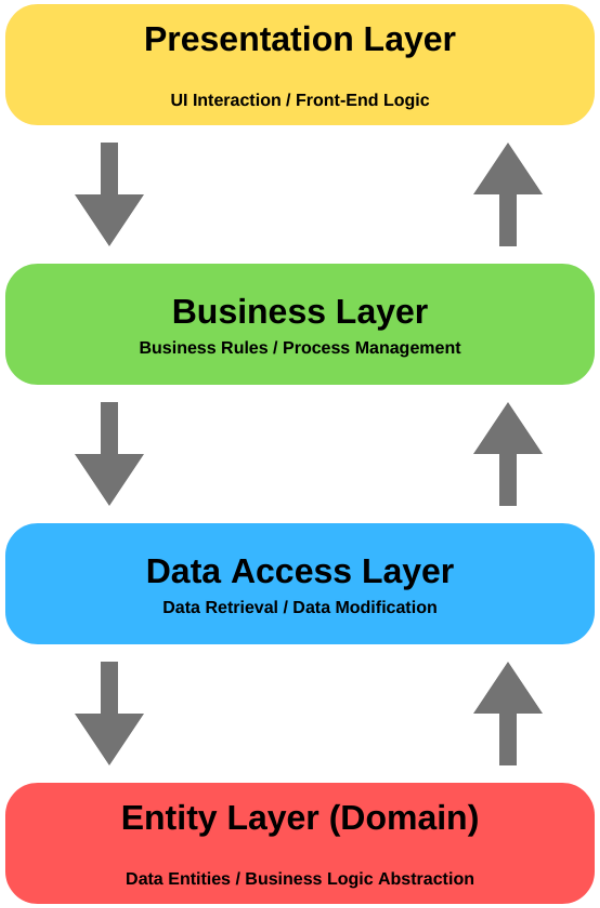
QUÉ QUEREMOS EVITAR?



ROTTEN DESIGN

ARQUITECTURA N CAPAS

Spring Boot y arquitectura n capas


Presentation Layer
UI Interaction / Front-End Logic
Business Layer
Business Rules / Process Management
Data Access Layer
Data Retrieval / Data Modification
Entity Layer (Domain)
Data Entities / Business Logic Abstraction

Spring
VS
Spring Boot
www.educba.com

HIBERNATE